1

SETTING UP A POWERSHELL TESTING ENVIRONMENT

In this chapter, you'll configure PowerShell so you can work through the code examples presented in the rest of the book. Then, we'll walk through a very quick overview of the PowerShell language, including its types, variables, and expressions. We'll also cover how to execute its commands, how to get help, and how to export data for later use.

Choosing a PowerShell Version

The most important tool you'll need to use this book effectively is PowerShell, which has been installed on the Windows operating system by default since Windows 7. However, there are many different versions of this tool. The version installed by default on currently supported versions of Windows is 5.1, which is suitable for our purposes, even though Microsoft no longer fully supports it. More recent versions of PowerShell are cross platform and open source but must be installed separately on Windows.

All the code presented in this book will run in both PowerShell 5.1 and the latest open source version, so it doesn't matter which you choose. If

you want to use the open source PowerShell, visit the project's GitHub page at **https://github.com/PowerShell/PowerShell** to find installation instructions for your version of Windows.

Configuring PowerShell

The first thing we need to do in PowerShell is set the *script execution policy*, which determines what types of scripts PowerShell can execute. For Windows clients running PowerShell 5.1, the default is `Restricted`, which blocks all scripts from running unless they are signed with a trusted certificate. As the scripts in this book are unsigned, we'll change the execution policy to `RemoteSigned`. This execution policy allows us to run unsigned PowerShell scripts if they're created locally but will not allow us to execute unsigned scripts downloaded in a web browser or attached to emails. Run the following command to set the execution policy:

```
PS> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force
```

The command changes the execution policy for the current user only, not the entire system. If you want to change it for all users, you'll need to start PowerShell as an administrator and then rerun the command, removing the `Scope` parameter.

If you're using the open source version of PowerShell or version 5.1 on Windows Server, then the default script execution policy is `RemoteSigned` and you do not need to change anything.

Now that we can run unsigned scripts, we can install the PowerShell module we'll be using for this book. A PowerShell *module* is a package of scripts and .NET binaries that export PowerShell commands. Every installation of PowerShell comes preinstalled with several modules for tasks ranging from configuring your applications to setting up Windows Update. You can install a module manually by copying its files, but the easiest approach is to use the PowerShell Gallery (**https://www.powershellgallery.com**), an online repository of modules.

To install a module from the PowerShell Gallery, we use PowerShell's `Install-Module` command. For this book, we'll need to install the

`NtObjectManager` module, which we can do using the following command:

```
PS> Install-Module NtObjectManager -Scope CurrentUser -Force
```

Make sure to say yes if the installer asks you any questions (after you've read and understood the question, of course). If you have the module installed already, you can ensure that you have the latest version by using the `Update-Module` command:

```
PS> Update-Module NtObjectManager
```

Once it's installed, you can load the module using the `Import-Module` command:

```
PS> Import-Module NtObjectManager
```

If you see any errors after importing the module, double-check that you've correctly set the execution policy; that's the most common reason for the module not loading correctly. As a final test, let's run a command that comes with the module to check that it's working. Execute the command in **Listing 1-1** and verify that the output matches what you see in the PowerShell console. We'll explore the purpose of this command in a later chapter.

```
PS> New-NtSecurityDescriptor
Owner DACL ACE Count SACL ACE Count Integrity Level
----- -------------- -------------- ---------------
NONE  NONE           NONE           NONE
```

Listing 1-1: Testing that the NtObjectManager module is working

If everything is working and you're comfortable with PowerShell, you can move on to the next chapter. If you need a quick refresher on the PowerShell language, keep reading.

An Overview of the PowerShell Language

A complete introduction to PowerShell is beyond the scope of this book. However, this section touches on various language features you'll need to be familiar with to use the book most effectively.

Understanding Types, Variables, and Expressions

PowerShell supports many different types, from basic integers and strings to complex objects. **Table 1-1** shows some of the most common built-in types, along with the underlying .NET runtime types and some simple examples.

Table 1-1: Common Basic PowerShell Types with .NET Types and Examples

Type	.NET type	Examples
`int`	`System.Int32`	`142`, `0x8E`, `0216`
`long`	`System.Int64`	`142L`, `0x8EL`, `0216L`
`string`	`System.String`	`"Hello"`, `'World!'`
`double`	`System.Double`	`1.0`, `1e10`
`bool`	`System.Boolean`	`$true`, `$false`
`array`	`System.Object[]`	`@(1, "ABC", $true)`
`hashtable`	`System.Collections.Hashtable`	`@{A=1; B="ABC"}`

To perform calculations on basic types, we can use well-known operators such as `+`, `-`, `*`, and `/`. These operators can be overloaded; for example, `+` is used for addition as well as for concatenating strings and arrays. **Table 1-2** provides a list of common operators, with simple examples and their results. You can test the examples yourself to check the output of each operator.

Table 1-2: Common Operators

Operator	Name	Examples	Results
`+`	Addition or concatenation	`1 + 2`, `"Hello" + "World!"`	`3`, `"HelloWorld!"`
`-`	Subtraction	`2 - 1`	`1`
`*`	Multiplication	`2 * 4`	`8`
`/`	Division	`8 / 4`	`2`
`%`	Modulus	`6 % 4`	`2`
`[]`	Index	`@(3, 2, 1, 0)[1]`	`2`
`-f`	String formatter	`"0x{0:X} {1}" -f 42, 123`	`"0x2A 123"`
`-band`	Bitwise AND	`0x1FF -band 0xFF`	`255`
`-bor`	Bitwise OR	`0x100 -bor 0x20`	`288`
`-bxor`	Bitwise XOR	`0xCC -bxor 0xDD`	`17`
`-bnot`	Bitwise NOT	`-bnot 0xEE`	`-239`
`-and`	Boolean AND	`$true -and $false`	`$false`
`-or`	Boolean OR	`$true -or $false`	`$true`
`-not`	Boolean NOT	`-not $true`	`$false`
`-eq`	Equals	`"Hello" -eq "Hello"`	`$true`

Operator	Name	Examples	Results
`-ne`	Not equals	`"Hello" -ne "Hello"`	`$false`
`-lt`	Less than	`4 -lt 10`	`$true`
`-gt`	Greater than	`4 -gt 10`	`$false`

You can assign values to variables using the assignment operator, `=`. A variable has an alphanumeric name prefixed with the `$` character. For example, **Listing 1-2** shows how you can capture an array in a variable and use the indexing operator to look up a value.

```
PS> $var = 3, 2, 1, 0
PS> $var[1]
2
```

Listing 1-2: Capturing an array in a variable and indexing it via the variable name

There are also some predefined variables we'll use in the rest of this book. These variables are:

`$null` Represents the NULL value, which indicates the absence of a value in comparisons

`$pwd` Contains the current working directory

`$pid` Contains the process ID of the shell

`$env` Accesses the process environment (for example, `$env:WinDir` to get the *Windows* directory)

You can enumerate all variables using the `Get-Variable` command.

In **Table 1-1**, you might have noticed that there were two string examples, one using double quotation marks and one using single quotation marks.

One difference between the two is that a double-quoted string supports *string interpolation*, where you insert a variable name into the string as a placeholder and PowerShell includes its value in the result. **Listing 1-3** shows what happens when you do this in double- and single-quoted strings.

```
PS> $var = 42
PS> "The magic number is $var"
The magic number is 42

PS> 'It is not $var'
It is not $var
```

Listing 1-3: Examples of string interpolation

First, we define a variable with the value `42` to insert into a string. Then we create a double-quoted string with the variable name inside it. The result is the string with the variable name replaced by its value formatted as a string. (If you want more control over the formatting, you can use the string formatter operator defined in **Table 1-2**.)

Next, to demonstrate the different behavior of a single-quoted string, we define one of these with the variable name inline. We can observe that in this case the variable name is copied verbatim and is not replaced by the value.

Another difference is that a double-quoted string can contain character escapes that are ignored in single-quoted strings. These escapes use a similar syntax to those of the C programming language, but instead of a backslash character (\) PowerShell uses the backtick (`). This is because Windows uses the backslash as a path separator, and writing out filepaths would be very annoying if you had to escape every backslash. **Table 1-3** gives a list of character escapes you can use in PowerShell.

Table 1-3: String Character Escapes

Character escape	Name
`` `0 ``	NUL character, with a value of zero

Character escape	Name
`` `a ``	Bell
`` `b ``	Backspace
`` `n ``	Line feed
`` `r ``	Carriage return
`` `t ``	Horizontal tab
`` `v ``	Vertical tab
`` `` ```	Backtick character
`` `" ``	Double quote character

If you want to insert a double quote character into a double-quoted string, you'll need to use the `` `" `` escape. To insert a single quote into a single- quoted string, you double the quote character: for example, `'Hello''There'` would convert to `Hello'There`. Note also the mention of a NUL character in this table. As PowerShell uses the .NET string type, it can contain embedded NUL characters. Unlike in the C language, adding a NUL will not terminate the string prematurely.

Because all values are .NET types, we can invoke methods and access properties on an object. For example, the following calls the `ToCharArray` method on a string to convert it to an array of single characters:

```
PS> "Hello".ToCharArray()
H
e
l
l
o
```

We can use PowerShell to construct almost any .NET type. The simplest way to do this is to cast a value to that type by specifying the .NET type in square brackets. When casting, PowerShell will try to find a suitable constructor for the type to invoke. For example, the following command will convert a string to a `System.Guid` object; PowerShell will find a constructor that accepts a string and call it:

```
PS> [System.Guid]"6c0a3a17-4459-4339-a3b6-1cdb1b3e8973"
```

You can also call a constructor explicitly by calling the `new` method on the type. The previous example can be rewritten as follows:

```
PS> [System.Guid]::new("6c0a3a17-4459-4339-a3b6-1cdb1b3e8973")
```

This syntax can also be used to invoke static methods on the type. For example, the following calls the `NewGuid` static method to create a new random globally unique identifier (GUID):

```
PS> [System.Guid]::NewGuid()
```

You can create new objects too, using the `New-Object` command:

```
PS> New-Object -TypeName Guid -ArgumentList "6c0a3a17-4459-4339-a3b6-1cdb1b3e897
```

This example is equivalent to the call to the static `new` function.

Executing Commands

Almost all commands in PowerShell are named using a common pattern: a verb and a noun, separated by a dash. For example, consider the command `Get-Item`. The `Get` verb implies retrieving an existing resource, while `Item` is the type of resource to return.

Each command can accept a list of parameters that controls the behavior of the command. For example, the `Get-Item` command accepts a `Path` parameter that indicates the existing resource to retrieve, as shown here:

```
PS> Get-Item -Path "C:\Windows"
```

The `Path` parameter is a *positional* parameter. This means that you can omit the name of the parameter, and PowerShell will do its best to select the best match. So, the previous command can also be written as the following:

```
PS> Get-Item "C:\Windows"
```

If a parameter takes a string value, and the string does not contain any special characters or whitespace, then you do not need to use quotes around the string. For example, the `Get-Item` command would also work with the following:

```
PS> Get-Item C:\Windows
```

The output of a single command is zero or more values, which can be basic or complex object types. You can pass the output of one command to another as input using a *pipeline*, which is represented by a vertical bar character, `|`. We'll see examples of using a pipeline when we discuss filtering, grouping, and sorting later in this chapter.

You can capture the result of an entire command or pipeline into a variable, then interact with the results. For example, the following captures the result of the `Get-Item` command and queries for the `FullName` property:

```
PS> $var = Get-Item -Path "C:\Windows"
PS> $var.FullName
C:\Windows
```

If you don't want to capture the result in a variable, you can enclose the command in parentheses and directly access its properties and methods:

```
PS> (Get-Item -Path "C:\Windows").FullName
C:\Windows
```

The length of a command line is effectively infinite. However, you'll want to try to split up long lines to make the commands more readable. The shell will automatically split a line on the pipe character. If you need to split a long line with no pipes, you can use the backtick character, then start a new line. The backtick must be the last character on the line; otherwise, an error will occur when the script is parsed.

Discovering Commands and Getting Help

A default installation of PowerShell has hundreds of commands to choose from. This means that finding a command to perform a specific task can be difficult, and even if you find the command, it might not be clear how to use it. To help, you can use two built-in commands, `Get-Command` and `Get-Help`.

The `Get-Command` command can be used to enumerate all the commands available to you. In its simplest form, you can execute it without any parameters and it will print all commands from all modules. However, it's probably more useful to filter on a specific word you're interested in. For example, **Listing 1-4** will list only the commands with the word `SecurityDescriptor` in their names.

```
PS> Get-Command -Name *SecurityDescriptor*
CommandType     Name                                Source
-----------     ----                                ------
Function        Add-NtSecurityDescriptorControl     NtObjectManager
Function        Add-NtSecurityDescriptorDaclAce     NtObjectManager
Function        Clear-NtSecurityDescriptorDacl      NtObjectManager
Function        Clear-NtSecurityDescriptorSacl      NtObjectManager
--snip--
```

Listing 1-4: Using Get-Command to enumerate commands

This command uses *wildcard syntax* to list only commands whose names include the specified word. Wildcard syntax uses a `*` character to represent any character or series of characters. Here, we've put the `*` on both sides of `SecurityDescriptor` to indicate that any text can come before or after it.

You can also list the commands available in a module. For example, **Listing 1-5** will list only the commands that are exported by the `NtObjectManager` module and begin with the verb `Start`.

```
PS> Get-Command -Module NtObjectManager -Name Start-*
CommandType     Name                                Source
-----------     ----                                ------
Function        Start-AccessibleScheduledTask       NtObjectManager
Function        Start-NtFileOplock                  NtObjectManager
Function        Start-Win32ChildProcess             NtObjectManager
Cmdlet          Start-NtDebugWait                   NtObjectManager
Cmdlet          Start-NtWait                        NtObjectManager
```

Listing 1-5: Using Get-Command to enumerate commands in the NtObjectManager module

Once you've found a command that looks promising, you can use the `Get-Help` command to inspect its parameters and get some usage examples. In **Listing 1-6**, we take the `Start-NtWait` command from **Listing 1-5** and pass it to `Get-Help`.

```
PS> Get-Help Start-NtWait
NAME
 ❶ Start-NtWait
SYNOPSIS
 ❷ Wait on one or more NT objects to become signaled.
SYNTAX
 ❸ Start-NtWait [-Object] <NtObject[]> [-Alertable <SwitchParameter>]
    [-Hour <int>] [-MilliSecond <long>]
    [-Minute <int>] [-Second <int>] [-WaitAll <SwitchParameter>]
    [<CommonParameters>]

    Start-NtWait [-Object] <NtObject[]> [-Alertable <SwitchParameter>]
    [-Infinite <SwitchParameter>] [-WaitAll <SwitchParameter>]
    [<CommonParameters>]
DESCRIPTION
 ❹ This cmdlet allows you to issue a wait on one or more NT
    objects until they become signaled.
--snip--
```

Listing 1-6: Displaying help for the Start-NtWait command

By default, `Get-Help` outputs the name of the command ❶, a short synopsis ❷, the syntax of the command ❸, and a more in-depth description ❹. In the command syntax section, you can see its multiple possible modes of operation: in this case, either specifying a time in hours, minutes, seconds, and/or milliseconds, or specifying `Infinite` to wait indefinitely.

When any part of the syntax is shown in brackets, `[]`, that means it's optional. For example, the only required parameter is `Object`, which takes an array of `NtObject` values. Even the name of this parameter is optional, as `-Object` is in brackets.

You can get more information about a parameter by using the `Parameter` command. **Listing 1-7** shows the details for the `Object` parameter.

```
PS> Get-Help Start-NtWait -Parameter Object
-Object <NtObject[]>
    Specify a list of objects to wait on.

    Required?                    true
    Position?                    0
    Default value
    Accept pipeline input?       true (ByValue)
    Accept wildcard characters?  False
```

Listing 1-7: Querying the details of the Object parameter with the Parameter command

You can use wildcard syntax to select a group of similar parameter names. For example, if you specify `Obj*`, then you'll get information about any parameters whose names start with the `Obj` prefix.

If you want usage examples for a command, use the `Examples` parameter, as demonstrated in **Listing 1-8**.

```
PS> Get-Help Start-NtWait -Examples
--snip--
    ----------  EXAMPLE 1  ----------
```

```
❶ $ev = Get-NtEvent \BaseNamedObjects\ABC
   Start-NtWait $ev -Second 10

❷ Get an event and wait for 10 seconds for it to be signaled.
--snip--
```

Listing 1-8: Showing examples for Start-NtWait

Each example should include a one- or two-line snippet of a PowerShell script ❶ and a description of what it does ❷. You can also see the full help output for the command by specifying the `Full` parameter. To view this output in a separate pop-up window, use the `ShowWindow` parameter. For example, try running this command:

```
PS> Get-Help Start-NtWait -ShowWindow
```

You should see the dialog shown in **Figure 1-1**.

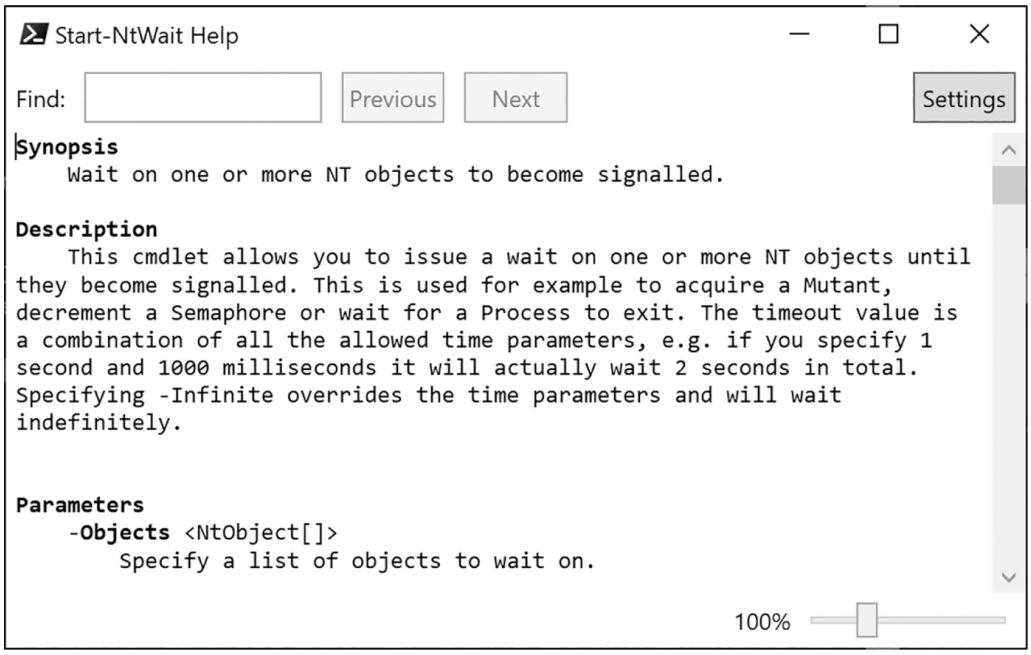


Figure 1-1: A dialog showing Get-Help information using the ShowWindow parameter

One final topic to mention about commands is that you can set up *aliases*, or alternative names for the commands. For example, you can use an alias to make commands shorter to type. PowerShell comes with many aliases predefined, and you can define your own using the `New-Alias` com-

mand. For example, we can set the `Start-NtWait` command to have the alias `swt` by doing the following:

```
PS> New-Alias -Name swt -Value Start-NtWait
```

To display a list of all the defined aliases, use the `Get-Alias` command. We'll avoid using aliases unnecessarily throughout this book, as it can make the scripts more confusing if you don't know what an alias represents.

Defining Functions

As with all programming languages, it pays to reduce complexity in PowerShell. One way of reducing complexity is to combine common code into a function. Once a function is defined, the PowerShell script can call the function rather than needing to repeat the same code in multiple places. The basic function syntax in PowerShell is simple; **Listing 1-9** shows an example.

```
PS> function Get-NameValue {
    param(
        [string]$Name = "",
        $Value
    )
    return "We've got $Name with value $Value"
}

PS> Get-NameValue -Name "Hello" -Value "World"
We've got Hello with value World

PS> Get-NameValue "Goodbye" 12345
We've got Goodbye with value 12345
```

Listing 1-9: Defining a simple PowerShell function called Get-NameValue

The syntax for defining a function starts with the keyword `function` followed by the name of the function you want to define. While it's not required to use the standard PowerShell command naming convention of a

verb followed by a noun, it pays to do so, as it makes it clear to the user what your function does.

Next, you define the function's named parameters. Like variables, parameters are defined using a name prefixed with `$`, as you can see in **Listing 1-9**. You can specify a type in brackets, but this is optional; in this example, `$Name` is a string, but the `$Value` parameter can take any value from the caller. Specifying named parameters is not required. If no `param` block is included, then any passed arguments are placed in the `$args` array. The first parameter is located at `$args[0]`, the second at `$args[1]`, and so on.

The body of the `Get-NameValue` function takes the parameters and builds a string using string interpolation. The function returns the string using the `return` keyword, which also immediately finishes the function. You can omit the `return` keyword in this case, as PowerShell will return any values uncaptured in variables.

After defining the function, we invoke it. You can specify the parameter names explicitly. However, if the call is unambiguous, then specifying the parameter names is not required. **Listing 1-9** shows both approaches.

If you want to run a small block of code without defining a function, you can create a script block. A *script block* is one or more statements enclosed in braces, `{}`. This block can be assigned to a variable and executed when needed using the `Invoke-Command` command or the `&` operator, as shown in **Listing 1-10**.

```
PS> $script = {Write-Output "Hello"}
PS> & $script
Hello
```

Listing 1-10: Creating a script block and executing it

Displaying and Manipulating Objects

If you execute a command and do not capture the results in a variable, the results are passed to the PowerShell console. The console will use a formatter to display the results, in either a table or a list (the format is

chosen automatically depending on the types of objects contained in the results). It's also possible to specify custom formatters. For example, if you use the built-in `Get-Process` command to retrieve the list of running processes, PowerShell uses a custom formatter to display the entries as a table, as shown in **Listing 1-11**.

```
PS> Get-Process
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    476      27    25896      32044       2.97   3352   1 ApplicationFrameHost
    623      18    25096      18524     529.95  19424   0 audiodg
    170       8     6680       5296       0.08   5192   1 bash
    557      31    23888        332       0.59  10784   1 Calculator
--snip--
```

Listing 1-11: Outputting the process list as a table

If you want to reduce the number of columns in the output, you can use the `Select-Object` command to select only the properties you need. For example, **Listing 1-12** selects the `Id` and `ProcessName` properties.

```
PS> Get-Process | Select-Object Id, ProcessName
   Id ProcessName
   -- -----------
 3352 ApplicationFrameHost
19424 audiodg
 5192 bash
10784 Calculator
--snip--
```

Listing 1-12: Selecting only the Id and ProcessName properties

You can change the default behavior of the output by using the `Format-Table` or `Format-List` command, which will force table or list formatting, respectively. For example, **Listing 1-13** shows how to use the `Format-List` command to change the output to a list.

```
PS> Get-Process | Format-List
Id      : 3352
```

```
Handles : 476
CPU     : 2.96875
SI      : 1
Name    : ApplicationFrameHost
--snip--
```

Listing 1-13: Using Format-List to show processes in a list view

To find the names of the available properties, you can use the `Get-Member` command on one of the objects that `Get-Process` returns. For example, **Listing 1-14** lists the properties of the `Process` object.

```
PS> Get-Process | Get-Member -Type Property
   TypeName: System.Diagnostics.Process
Name                MemberType Definition
----                ---------- ----------
BasePriority        Property   int BasePriority {get;}
Container           Property   System.ComponentModel.IContainer Container {get;}
EnableRaisingEvents Property   bool EnableRaisingEvents {get;set;}
ExitCode            Property   int ExitCode {get;}
ExitTime            Property   datetime ExitTime {get;}
--snip--
```

Listing 1-14: Using the Get-Member command to list properties of the Process object

You might notice that there are other properties not included in the output. To display them, you need to override the custom formatting. The simplest way to access the hidden properties is to use `Select-Object` to extract the values explicitly, or specify the properties to display to the `Format-Table` or `Format-List` command. You can use `*` as a wildcard to show all properties, as in **Listing 1-15**.

```
PS> Get-Process | Format-List *
Name            : ApplicationFrameHost
Id              : 3352
PriorityClass   : Normal
FileVersion     : 10.0.18362.1 (WinBuild.160101.0800)
HandleCount     : 476
WorkingSet      : 32968704
```

```
PagedMemorySize : 26517504
--snip--
```

Listing 1-15: Showing all the properties of the Process object in a list

Many objects also have methods you can call to perform some action on the object. **Listing 1-16** shows how you can use `Get-Member` to query for methods.

```
PS> Get-Process | Get-Member -Type Method
   TypeName: System.Diagnostics.Process

Name                 MemberType Definition
----                 ---------- ----------
BeginErrorReadLine   Method     void BeginErrorReadLine()
BeginOutputReadLine Method     void BeginOutputReadLine()
CancelErrorRead      Method     void CancelErrorRead()
CancelOutputRead     Method     void CancelOutputRead()
Close                Method     void Close()
--snip--
```

Listing 1-16: Displaying the methods on a Process object

If the output from a command is too long to fit on the screen, you can *page* the output so that only the first part is displayed, and the console will wait for you to press a key before displaying more. You can enable paging by piping the output to the `Out-Host` command and specifying the `Paging` parameter, or by using the `more` command. **Listing 1-17** shows an example.

```
PS> Get-Process | Out-Host -Paging
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    476      27    25896      32044       2.97   3352   1 ApplicationFrameHost
    623      18    25096      18524     529.95  19424   0 audiodg
    170       8     6680       5296       0.08   5192   1 bash
    557      31    23888        332       0.59  10784   1 Calculator
<SPACE> next page; <CR> next line; Q quit
```

Listing 1-17: Paging output using Out-Host

You can write directly to the console window by using the `Write-Host` command in your own scripts. This allows you to change the colors of the output to suit your taste, using the `ForegroundColor` and `BackgroundColor` parameters. It also has the advantage of not inserting objects into the pipeline by default, as shown here:

```
PS> $output = Write-Host "Hello"
Hello
```

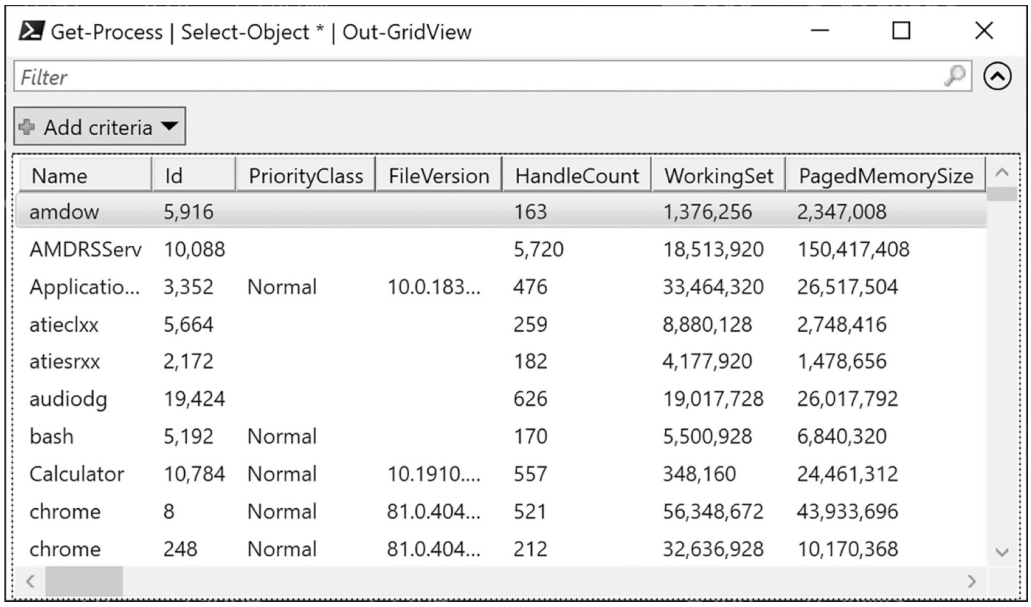
This means that, by default, you can't redirect the output to a file or into a pipeline. However, you can redirect the host output by redirecting its stream to the standard output stream using a command like the following:

```
PS> $output = Write-Host "Hello" 6>&1
PS> $output
Hello
```

PowerShell also supports a basic GUI to display tables of objects. To access it, use the `Out-GridView` command. Note that the custom formatting will still restrict what columns PowerShell displays. If you want to view other columns, use `Select-Object` in the pipeline to select the properties. The following example displays all properties in the Grid View GUI:

```
PS> Get-Process | Select-Object * | Out-GridView
```

Running this command should show a dialog like **Figure 1-2**.



Name	Id	PriorityClass	FileVersion	HandleCount	WorkingSet	PagedMemorySize
amdow	5,916			163	1,376,256	2,347,008
AMDRSServ	10,088			5,720	18,513,920	150,417,408
Applicatio...	3,352	Normal	10.0.183...	476	33,464,320	26,517,504
atieclxx	5,664			259	8,880,128	2,748,416
atiesrxx	2,172			182	4,177,920	1,478,656
audiodg	19,424			626	19,017,728	26,017,792
bash	5,192	Normal		170	5,500,928	6,840,320
Calculator	10,784	Normal	10.1910....	557	348,160	24,461,312
chrome	8	Normal	81.0.404...	521	56,348,672	43,933,696
chrome	248	Normal	81.0.404...	212	32,636,928	10,170,368

Figure 1-2: Showing Process objects in a grid view

You can filter and manipulate the data in the Grid View GUI. Try playing around with the controls. You can also specify the `PassThru` parameter to `Out-GridView`, which causes the command to wait for you to click the OK button in the GUI. Any rows in the view that are selected when you click OK will be written to the command pipeline.

Filtering, Ordering, and Grouping Objects

A traditional shell passes raw text between commands; PowerShell passes objects. Passing objects lets you access individual properties of the objects and trivially filter the pipeline. You can even order and group the objects easily.

You can filter objects using the `Where-Object` command, which has the aliases `Where` and `?`. The simplest filter is to check for the value of a parameter, as shown in **Listing 1-18**, where we filter the output from the built-in `Get-Process` command to find the `explorer` process.

```
PS> Get-Process | Where-Object ProcessName -EQ "explorer"
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
   2792     130   118152     158144     624.83    6584   1 explorer
```

Listing 1-18: Filtering a list of processes using Where-Object

In **Listing 1-18**, we pass through only `Process` objects where the `ProcessName` equals (`-EQ`) `"explorer"`. There are numerous operators you can use for filtering, some of which are shown in **Table 1-4**.

Table 1-4: Common Operators for `Where-Object`

Operator	Example	Description
`-EQ`	`ProcessName -EQ "explorer"`	Equal to the value
`-NE`	`ProcessName -NE "explorer"`	Not equal to the value
`-Match`	`ProcessName -Match "ex.*"`	Matches a string against a regular expression
`-NotMatch`	`ProcessName -NotMatch "ex.*"`	Inverse of the `-Match` operator
`-Like`	`ProcessName -Like "ex*"`	Matches a string against a wildcard
`-NotLike`	`ProcessName -NotLike "ex*"`	Inverse of the `-Like` operator
`-GT`	`ProcessName -GT "ex"`	Greater-than comparison
`-LT`	`ProcessName -LT "ex"`	Less-than comparison

You can investigate all of the supported operators by using `Get-Help` on the `Where-Object` command. If the condition to filter on is more complex than a simple comparison, you can use a script block. The script block should return `True` to keep the object in the pipeline or `False` to filter it. For example, you could also write **Listing 1-18** as the following:

```
PS> Get-Process | Where-Object {$_.ProcessName -eq "explorer"}
```

The `$_` variable passed to the script block represents the current object in the pipeline. By using a script block you can access the entire language in your filtering, including calling functions.

To order objects, use the `Sort-Object` command. If the objects can be ordered, as in the case of strings or numbers, then you just need to pipe the objects into the command. Otherwise, you'll need to specify a property to sort on. For example, you can sort the process list by its handle count, represented by the `Handles` property, as shown in **Listing 1-19**.

```
PS> Get-Process | Sort-Object Handles
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
      0       0       60          8                0   0 Idle
     32       9     4436       6396               1032   1 fontdrvhost
     53       3     1148       1080                496   0 smss
     59       5      804       1764                908   0 LsaIso
--snip--
```

Listing 1-19: Sorting processes by the number of handles

To sort in descending order instead of ascending order, use the `Descending` parameter, as shown in **Listing 1-20**.

```
PS> Get-Process | Sort-Object Handles -Descending
Handles   NPM(K)    PM(K)      WS(K)     CPU(s)      Id   SI  ProcessName
-------  ------    -----      -----     ------      --   --  -----------
   5143       0       244      15916                 4    0  System
   2837     130    116844     156356     634.72    6584    1  explorer
   1461      21     11484      16384               1116    0  svchost
   1397      52     55448       2180      12.80   12452    1  Microsoft.Photos
```

Listing 1-20: Sorting processes by the number of handles in descending order

It's also possible to filter out duplicate entries at this stage by specifying the `Unique` parameter to `Sort-Object`.

Finally, you can group objects based on a property name using the `Group-Object` command. **Listing 1-21** shows that this command returns a list of

objects, each with `Count`, `Name`, and `Group` properties.

```
PS> Get-Process | Group-Object ProcessName
Count Name                  Group
----- ----                  -----
    1 ApplicationFrameHost {System.Diagnostics.Process (ApplicationFrameHost)}
    1 Calculator           {System.Diagnostics.Process (Calculator)}
   11 conhost              {System.Diagnostics.Process (conhost)...}
--snip--
```

Listing 1-21: Grouping Process objects by ProcessName

Alternatively, you could use all of these commands together in one pipeline, as shown in **Listing 1-22**.

```
PS> Get-Process | Group-Object ProcessName |
Where-Object Count -GT 10 | Sort-Object Count
Count Name                  Group
----- ----                  -----
   11 conhost              {System.Diagnostics.Process (conhost),...}
   83 svchost              {System.Diagnostics.Process (svchost),...}
```

Listing 1-22: Combining Where-Object, Group-Object, and Sort-Object

Exporting Data

Once you've got the perfect set of objects you want to inspect, you might want to persist that information to a file on disk. PowerShell provides numerous options for this, a few of which I'll discuss here. The first option is to output the objects to a file as text, using `Out-File`. This command captures the formatted text output and writes it to a file. You can use `Get-Content` to read the file back in again, as shown in **Listing 1-23**.

```
PS> Get-Process | Out-File processes.txt
PS> Get-Content processes.txt
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    476      27    25896      32044       2.97   3352   1 ApplicationFrameHost
    623      18    25096      18524     529.95  19424   0 audiodg
```

```
    170       8     6680       5296       0.08   5192   1 bash
    557      31    23888        332       0.59  10784   1 Calculator
--snip--
```

Listing 1-23: Writing content to a text file and reading it back in again

You can also use the greater-than operator to send the output to a file, as in other shells. For example:

```
PS> Get-Process > processes.txt
```

If you want a more structured format, you can use `Export-Csv` to convert the object to a comma-separated value (CSV) table format. You could then import this file into a spreadsheet program to analyze offline. The example in **Listing 1-24** selects some properties of the `Process` object and exports them to the CSV file *processes.csv*.

```
PS> Get-Process | Select-Object Id, ProcessName |
Export-Csv processes.csv -NoTypeInformation
PS> Get-Content processes.csv
"Id","ProcessName"
"3352","ApplicationFrameHost"
"19424","audiodg"
"5192","bash"
"10784","Calculator"
--snip--
```

Listing 1-24: Exporting objects to a CSV file

It's possible to reimport the CSV data using the `Import-Csv` command. However, if you expect to export the data and then reimport it later, you'll probably prefer the CLI XML format. This format can include the structure and type of the original object, which allows you to reconstruct it when you import the data. **Listing 1-25** shows how you can use the `Export-CliXml` and `Import-CliXml` commands to export objects in this format and then reimport them.

```
PS> Get-Process | Select-Object Id, ProcessName | Export-CliXml processes.xml
PS> Get-Content processes.xml
<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04">
  <Obj RefId="0">
    <TNRef RefId="0" />
    <MS>
      <I32 N="Id">3352</I32>
      <S N="ProcessName">ApplicationFrameHost</S>
    </MS>
  </Obj>
--snip--
</Objs>
PS> $ps = Import-CliXml processes.xml
PS> $ps[0]
  Id ProcessName
  -- -----------
3352 ApplicationFrameHost
```

Listing 1-25: Exporting and reimporting CLI XML files

This concludes our discussion of the PowerShell language. If you're a little rusty, I recommend picking up a good book on the topic, such as *PowerShell for Sysadmins* by Adam Bertram (No Starch Press, 2020).

Wrapping Up

This chapter gave a short overview of how to set up your PowerShell environment so that you can run the code examples included throughout the book. We discussed configuring PowerShell to run scripts and installing the required external PowerShell module.

The rest of the chapter provided a bit of background on the PowerShell language. This included the basics of PowerShell syntax, as well as discovering commands using `Get-Command`, getting help using `Get-Help`, and displaying, filtering, grouping, and exporting PowerShell objects.

With the basics of PowerShell out of the way, we can start to dive into the inner workings of the Windows operating system. In the next chapter, we'll discuss the Windows kernel and how you can interact with it using PowerShell.